

README Frontend

Responsable documentacion frontend: Hector Garcia - G8.

Resumen

El frontend es una aplicacion multipagina servida directamente por Spring Boot desde `src/main/resources/static`. No usa framework de cliente pesado; se apoya en HTML, Bootstrap 5, CSS comun y JavaScript vanilla organizado por perfil.

Tecnologias

- HTML5.
- Bootstrap 5.
- CSS propio en `css/styles.css`.
- JavaScript vanilla.
- Fetch API para comunicacion con el backend.
- Sesion de Spring Security mediante cookie.

Estructura

```
src/main/resources/static
|-- admin          Pantallas de administracion
|-- auxiliar       Pantallas de recepcion/auxiliar
|-- cliente        Portal privado del cliente
|-- css            Estilos comunes
|-- error          Paginas de error
|-- js
|   |-- admin
|   |-- auxiliar
|   |-- cliente
|   |-- common
|   |-- veterinario
|-- veterinario    Pantallas clinicas
|-- index.html
|-- login.html
|-- register.html
```

Archivos comunes

js/common/api.js

Centraliza llamadas al backend:

- apiGet .
- apiPost .
- apiPut .
- apiDelete .
- showStatus .
- escapeHtml .
- formatDate .
- estadoBadge .

Tambien interpreta errores JSON del backend, incluidos mensajes de validacion por campo.

js/common/layout.js

Construye la navegacion comun segun el atributo `data-role` del `body` . Esto evita repetir menus completos en cada pantalla.

`js/common/auth.js`

Gestiona el formulario de registro publico. Envia nombre, apellidos, DNI, telefono, direccion, email, usuario y contrasena a `/api/auth/register` .

Pantallas por perfil

Publicas

Pantalla	Funcion
<code>/index.html</code>	Entrada publica del proyecto.
<code>/login.html</code>	Inicio de sesion.
<code>/register.html</code>	Alta de cliente.

Administrador

Pantalla	Funcion
<code>/admin/dashboard.html</code>	Resumen del area admin.
<code>/admin/usuarios.html</code>	Gestion de usuarios.
<code>/admin/perfiles.html</code>	Consulta de roles/perfiles.

Auxiliar

Pantalla	Funcion
<code>/auxiliar/dashboard.html</code>	Resumen de recepcion.
<code>/auxiliar/clientes.html</code>	Alta, edicion, busqueda y baja de clientes.
<code>/auxiliar/mascotas.html</code>	Gestion de mascotas.
<code>/auxiliar/citas.html</code>	Agenda y confirmacion de citas.
<code>/auxiliar/reiac.html</code>	Verificacion simulada de microchip.

Veterinario

Pantalla	Funcion
/veterinario/dashboard.html	Resumen del area veterinaria.
/veterinario/lista-espera.html	Citas confirmadas pendientes de atender.
/veterinario/consulta-form.html	Registro de consulta.
/veterinario/recetas.html	Creacion de recetas.

Cliente

Pantalla	Funcion
/cliente/dashboard.html	Resumen privado.
/cliente/citas.html	Solicitud de citas en huecos disponibles.
/cliente/estado.html	Estado de citas y consultas.
/cliente/historial.html	Historial clinico.
/cliente/pagos.html	Pagos pendientes y realizados.

Flujo de citas en frontend

1. El cliente abre /cliente/citas.html .
2. El frontend carga sus mascotas desde /api/portal/mascotas .
3. El frontend carga veterinarios desde /api/veterinarios .
4. Al elegir veterinario, carga huecos desde /api/portal/citas/disponibles?veterinarioId=...
5. El cliente solicita cita con POST /api/portal/citas .
6. La cita aparece como PENDIENTE .
7. El auxiliar la confirma desde /auxiliar/citas.html .
8. El veterinario la ve en lista de espera cuando esta CONFIRMADA .

Responsive

El proyecto usa:

- `meta viewport` en las paginas.
- Grid de Bootstrap.
- CSS responsive para contenedores, paneles, tablas y formularios.
- Navegacion lateral que se adapta a pantalla pequena.

Criterios de calidad aplicados

- Las paginas protegidas se separan por rol.
- Los formularios muestran errores devueltos por backend.
- Las tablas escapan contenido dinamico con `escapeHtml`.
- Los estados se muestran con badges.
- El CSS comun mantiene una identidad visual consistente.
- No se duplican menus completos gracias a `layout.js`.